TEC | Tecnológico de Costa Rica

Proyecto #1

IC-3002 - Análisis de Algoritmos

Recorrido del caballo

Estudiantes:

Tayler Wynta Rodríguez

Maikel Flores Navarro

Axel López Cruz

Profesor:

Joss Pecou Johnson

II Semestre 2025

Descripción del algoritmo

El módulo logica.py implementa una solución del problema *Knight's Tour* (recorrido del caballo) usando **backtracking recursivo**. Permite buscar tanto recorridos **abiertos** (el caballo visita todas las casillas exactamente una vez; no se exige volver al inicio) como **cerrados** (al terminar, el caballo puede volver al punto inicial en un solo movimiento). La función principal generar_ruta_caballo(...) encapsula el proceso y devuelve la trayectoria encontrada (si existe) junto con el tiempo de cómputo medido en segundos.

Visión General del Algoritmo

Representación del tablero

- El tablero N×N se representa como una lista de listas (tablero).
- Cada celda contiene:
  - -1 → no visitada.
  - 0,1,2,... → número de movimiento asignado.

Movimientos del caballo

- Desde una casilla (r,c) el caballo puede moverse a 8 posiciones en "L": (±2, ±1) y (±1, ±2).
- Antes de mover se verifica con Movimiento_Valido que la casilla destino esté dentro del tablero y no haya sido visitada (== -1).

Búsqueda por backtracking

- Se exploran recursivamente los movimientos válidos.
- Al aplicar un movimiento, se marca la casilla con el número secuencial de paso y se añade un evento de avance a Ruta.
- Si la llamada recursiva no conduce a solución, se restaura el estado (desmarcar la casilla) y se añade un evento de retroceso a Ruta.

Condición de éxito

- Si contador == N * N (se visitaron todas las casillas), en:
    - recorrido abierto: se guarda un snapshot del tablero (se añade con copy.deepcopy(tablero)) y se retorna éxito.
    - recorrido cerrado: además se verifica que la última posición esté en la lista de posiciones adyacentes a la inicial (generada por Encontrar_Soluciones); sólo entonces se guarda y se retorna con éxito.

Registro (Ruta)

- Ruta es la lista que el algoritmo va llenando para la visualización:
    - contiene tuplas (fila, columna, tipo) donde tipo es 'A' (avance) o 'R' (retroceso).
    - además, en el caso de encontrar solución completa, se añade un estado completo del tablero (copy.deepcopy(tablero)) como último elemento.

Medición de tiempo

- generar_ruta_caballo mide el tiempo de ejecución con time.perf_counter() y devuelve tiempo_total (segundos) junto con Ruta.

Creación del tablero — función CrearTablero(Tamaño, FilaInicial, ColumnaInicial)

Esta función construye la matriz inicial que representa el tablero de ajedrez y coloca el caballo en su posición de partida. El tablero usa el valor -1 para indicar casillas no visitadas y marca la casilla inicial con 0, que representa el primer movimiento del caballo.

La función CrearTablero:

- Crea una lista de listas de tamaño Tamaño × Tamaño e inicializa todas las celdas a -1.

- Coloca el valor 0 en la posición (FilaInicial, ColumnaInicial) para indicar la posición inicial del caballo.
- Devuelve la matriz resultante para que el algoritmo de búsqueda la modifique durante la exploración.

Validación de movimiento — función Movimiento_Valido(tablero, filaActual, colActual, Tamaño)

Comprueba si una posición concreta es válida para que el caballo se desplace: la celda debe estar dentro de los límites del tablero y no haber sido visitada (valor -1).

La función Movimiento_Valido:

- Verifica que filaActual y colActual estén dentro del rango [0, Tamaño-1].
- Comprueba que tablero[filaActual][colActual] == -1, es decir, que la casilla no ha sido visitada.
- Devuelve True si ambas condiciones se cumplen; False en cualquier otro caso.

Búsqueda de camino abierto — función Encontrar_Camino_Abierto(filaActual, colActual, tablero, Tamaño, contador, Ruta)

Realiza la búsqueda recursiva por backtracking para encontrar un recorrido abierto del caballo (no requiere que el último movimiento vuelva al inicio). Explora sistemáticamente los 8 desplazamientos posibles desde la casilla actual, aplicando movimientos válidos y retrocediendo cuando ninguna rama conduce a una solución completa.

La función Encontrar_Camino_Abierto:

- Comprueba si contador == Tamaño * Tamaño (condición de éxito: todas las casillas visitadas). En ese caso marca la casilla final con 'C', añade el tablero a Ruta y devuelve True.

- Para cada uno de los 8 movimientos posibles (orden fijo en el código) verifica con Movimiento_Valido y, si es válido, delega la ejecución al auxiliar realizar_movimiento_Abierto.
- Si alguna de las llamadas recursivas devuelve True, propaga True hacia arriba; si todas fallan, retorna False permitiendo el retroceso del nivel anterior.

Aplicación de un paso (abierto) — función realizar_movimiento_Abierto(tablero, filaActual, colActual, Tamaño, contador, filaAnterior, colAnterior, Ruta)

Auxiliar que aplica un movimiento en la búsqueda de recorrido abierto: marca la casilla destino, registra el avance, llama recursivamente a Encontrar_Camino_Abierto y, si la rama no conduce a solución, restaura el tablero y registra el retroceso.

La función realizar_movimiento_Abierto:

- Escribe en tablero[filaAnterior][colAnterior] el valor contador - 1 (conservar numeración anterior).
- Marca la casilla destino con contador y añade (filaActual, colActual, 'A') en Ruta.
- Llama a Encontrar_Camino_Abierto con contador + 1. Si devuelve True, retorna True.
- Si la recursión falla, desmarca la casilla destino (-1), restaura la casilla anterior a contador - 1 y añade (filaAnterior, colAnterior, 'R') a Ruta, retornando False.

Búsqueda de camino cerrado — función Encontrar_Camino_Cerrado(filaActual, colActual, tablero, Tamaño, contador, soluciones, Ruta)

Equivalente a la búsqueda abierta pero con la condición adicional de cierre: la posición final, una vez visitadas todas las casillas, debe estar en la lista soluciones (posiciones desde las que se puede volver al punto inicial en un solo movimiento).

La función Encontrar_Camino_Cerrado:

- Si contador == Tamaño * Tamaño, comprueba si (filaActual, colActual) está en soluciones. Si es así marca 'C', añade a Ruta y retorna True; si no, retorna False.
- Para cada movimiento válido llama a Realizar_Movimiento_Cerrado; si alguna llamada devuelve True, la función retorna True.
- Si ningún movimiento conduce a solución, retorna False.

Aplicación de un paso (cerrado) — función Realizar_Movimiento_Cerrado(tablero, filaActual, colActual, Tamaño, contador, soluciones, filaAnterior, colAnterior, Ruta)

Auxiliar para la variante cerrada; aplica el movimiento, registra avance en Ruta, invoca la recursión y, si falla, deshace el movimiento y registra retroceso.

La función Realizar_Movimiento_Cerrado:

- Marca la casilla anterior con contador - 1 y la casilla actual con contador.
- Añade (filaActual, colActual, 'A') a Ruta y llama a Encontrar_Camino_Cerrado con contador + 1.
- Si la recursión falla, restaura la casilla actual a -1, restaura la anterior a contador - 1 y añade (filaAnterior, colAnterior, 'R') a Ruta.

Cálculo de posiciones candidatas al cierre — función Encontrar_Soluciones(tablero, fila, columna, tamaño)

Genera la lista de posiciones alcanzables desde la casilla inicial que, si la búsqueda termina en ellas, permitirían volver al inicio en un solo movimiento (candidatas para un recorrido cerrado).

La función Encontrar_Soluciones:

- Evalúa las 8 posiciones en “L” desde (fila, columna) y añade aquellas que son válidas según Movimiento_Valido (están dentro del tablero y no visitadas).

- Devuelve la lista de tuplas (fila, columna) con las posiciones candidatas.

Función generar_ruta_caballo(tamaño, fila, columna, tipo)

Función de alto nivel que coordina la generación del tablero, lanza la búsqueda (abierta o cerrada), registra la ruta para visualización y mide el tiempo de cómputo.

La función generar_ruta_caballo:

- Inicia un contador temporal (time.perf_counter()) antes de comenzar la búsqueda.
- Crea el tablero inicial con CrearTablero y prepara Ruta con la entrada inicial (fila,columna,'A').
- Según tipo ('abierto' o 'cerrado') invoca la búsqueda correspondiente (Encontrar_Camino_Abierto o Encontrar_Camino_Cerrado).
- Si se encuentra solución, mide el tiempo total y devuelve (Ruta, tiempo_total). Si no hay solución devuelve ([], tiempo_total) con el tiempo consumido.

Instrucciones de ejecución

1. **Herramientas / Dependencias**
    a. Python (recomendado 3.10+)
    b. Flask
    c. Instala Flask ejecutando en la terminal:
    d. pip install Flask
2. **Descargar el repositorio**
    a. Ve al enlace: https://github.com/MaikelFn/Proyecto-Recorrido-del-Caballo-en-Ajedrez
    b. Haz clic en "Code" y selecciona "Download ZIP" para descargar el proyecto, o clónalo con Git: git clone https://github.com/MaikelFn/Proyecto-Recorrido-del-Caballo-en-Ajedrez.git

c. Descomprime el ZIP o abre el directorio descargado.

3. **Ejecutar el proyecto**
   a. Abre una terminal y navega al directorio del código: cd Proyecto-Recorrido-del-Caballo-en-Ajedrez/Codigo
   b. Ejecuta la aplicación Flask: python app.py
4. **Abrir la aplicación en el navegador**
   a. Una vez que Flask esté corriendo, abre tu navegador y visita: http://127.0.0.1:5000/

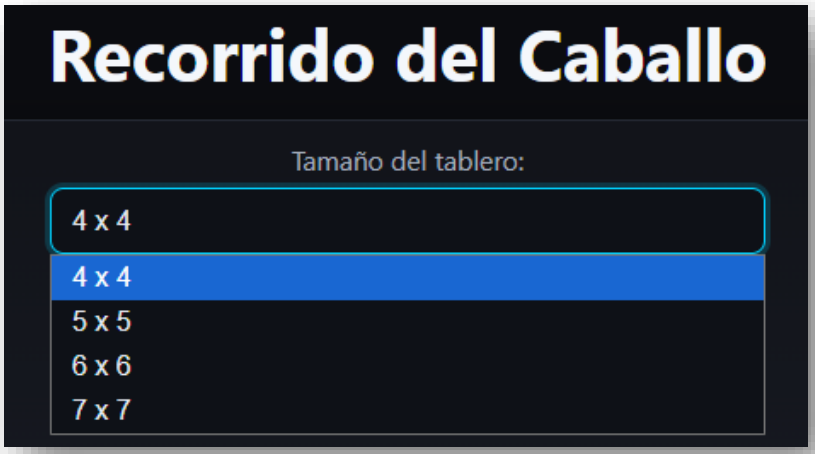
Manual de Usuario

1) Inicio

Tras ejecutar el código y abrir el enlace en consola, se mostrará esta interfaz inicial, en donde el usuario podrá seleccionar/introducir los parámetros para la simulación del recorrido del caballo.

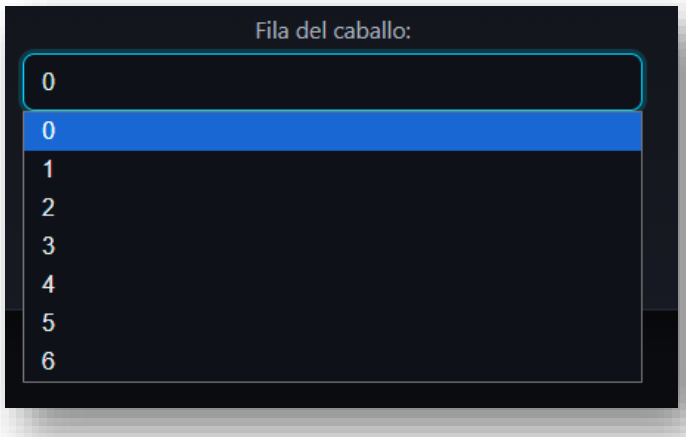


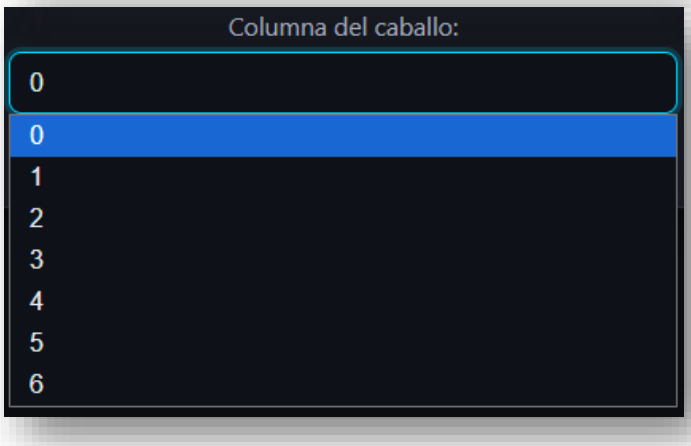
Inicialmente, el usuario deberá ingresar las dimensiones nxn de la matriz para la simulación del recorrido del caballo (El tablero).



2) Ingreso de parámetros

Una vez seleccionado, se debe ingresar la posición del caballo en la matriz (Solo se permiten valores de 0 hasta n-1, siendo n el valor de nxn de la matriz) siendo estos los valores i para filas y j para columnas.





3) Generación del tablero

Una vez seleccionados los parámetros iniciales para las dimensiones de la matriz y posicionamiento inicial del caballo, se presiona el botón “Generar”.



Una vez generada la matriz, se podrá visualizar el tablero y el caballo en la posición ingresada por el usuario, además de 3 botones “Inicio”, “Recorrido Abierto” y “Recorrido Cerrado”.

4) Selección de recorrido

Recorrido Abierto

El caballo parte de su posición inicial y recorre el tablero en movimientos en L, siendo cada movimiento en una casilla “p”, hacia una las 4 direcciones, Este, Oeste, Norte o Sur, 3 casillas y luego una posición n+1 o n-1 según lo necesario.

En caso de sí haber una solución, se mostrará esta interfaz, en donde se podrá “Iniciar Animación” o “Inicio”.



Al seleccionar “Iniciar Animación” se mostrará el recorrido paso por paso y se mostraran 2 botones “Ver Tablero Final” e Inicio.

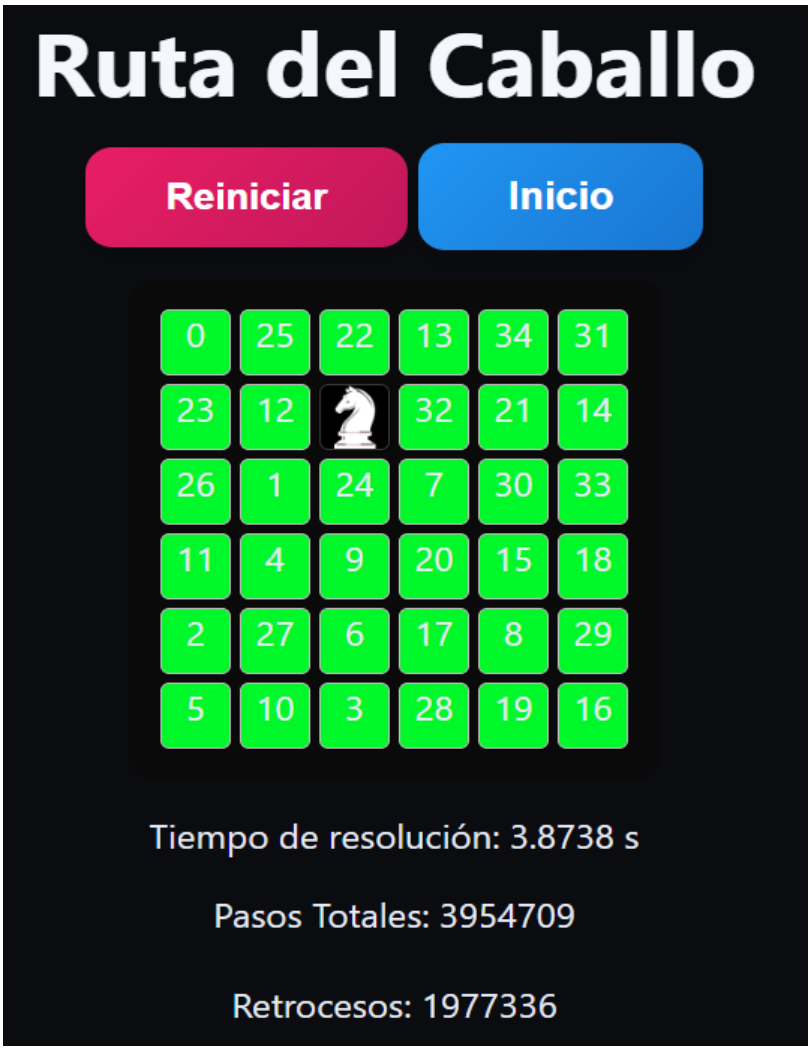


Al finalizar o al presionar “Ver Tablero Final” se verán las estadísticas generales del recorrido, así como el tablero recorrido en orden por enumeración de los movimientos realizados y casillas visitadas. Al “Reiniciar” se resolverá el mismo tablero con los mismos parámetros.



Recorrido Cerrado

El caballo parte de su posición inicial y recorre el tablero en movimientos en L, siendo cada movimiento en una casilla “p”, hacia una las 4 direcciones, Este, Oeste, Norte o Sur, 2 casillas y luego una posición adyacente n+1 o n-1 según lo necesario. A diferencia del recorrido abierto, el caballo debe finalizar su recorrido únicamente en cualquiera de sus casillas adyacentes.



5) Finalización de recorrido

Reiniciar

En ambos casos (recorrido abierto y cerrado) el botón de reiniciar cumple la misma funciona reiniciar el recorrido y volver a la ventana de Iniciar Animación



Inicio

En todas las ventanas en las que aparece este botón tiene la misma función y regresar a la ventana del menú principal.



Manejo de Errores

Durante las pruebas con diferentes tamaños de tablero, se identificaron varios casos donde el algoritmo no puede encontrar una solución válida para el recorrido del caballo. Estos errores no se deben necesariamente a fallos del programa, sino a limitaciones matemáticas o estructurales del propio tablero.

Tablero 4×4

- **Situación:** No existe ningún recorrido completo posible (ni abierto ni cerrado).
- **Motivo:** El tablero es demasiado pequeño; el caballo se queda sin movimientos antes de visitar todas las casillas.
- **Comportamiento en la app:** Si el usuario intenta generar cualquier recorrido, el sistema muestra una ventana de error indicando que no hay solución posible en 4×4, sin importar la posición inicial.

Tablero 5×5

- **Situación:** Solo se pueden hacer recorridos abiertos; los cerrados no existen.
- **Motivo:** Al ser un tablero de tamaño impar, hay una casilla más de un color que de otro, y el caballo siempre cambia de color al saltar. Por eso no puede volver al punto de partida.

Tablero 7×7

- **Situación:** Solo es posible el recorrido abierto. Los recorridos cerrados son imposibles.
- **Motivo:** Igual que en 5×5, el tamaño impar rompe la simetría necesaria para cerrar el recorrido.

7x7 y 5x5 si se selecciona el camino cerrado automáticamente muestra el mensaje de error.

Tamaño del tablero	Tipo de recorrido posible	Motivo de error principal
4×4	Ninguno	Tablero demasiado pequeño
5x5	Solo abierto	Tamaño impar (no se puede cerrado)
7x7	Solo abierto	Tamaño impar (no se puede cerrado)

Por estos motivos decidimos colocar una validación antes de iniciar el proceso en el caso de los tableros



- **Pruebas de tablero 4x4**
    a. Ejemplo



Como ya mencionamos previamente en un tablero 4x4 no existen caminos abiertos o cerrados sim importar la pocision de inicio.

- **Pruebas de tablero 5x5 (camino abierto)**
    b. Ejemplo



En este ejemplo podemos observar el resultado de un recorrido abierto partiendo desde la pocision central del tablero(2,2).

- **Pruebas de tablero 5x5 (camino cerrado)**
    a. Ejemplo



En esta imagen observamos el mensaje de error que se muestra al intentar realizar un recorrido cerrado con un tablero 5x5.

- **Pruebas de tablero 6x6 (camino abierto)**
    b. Ejemplo



Aquí podemos observar el recorrido abierto para un tablero 6x6 iniciando desde la pocision (0,0).

- **Pruebas de tablero 6x6 (camino cerrado)**
    c. Ejemplo



En este ejemplo observamos como el caballo quedó a una casilla de su pocision inicial (5,5) lo que cumple con el recorrido cerrado.

- **Pruebas de tablero 7x7 (camino abierto)**
    a. Ejemplo



Este ejemplo muestra la ruta abierta para una matriz 7x7 desde la pocision inicial (0,0).

Enlaces

Repositorio de Git: https://github.com/MaikelFn/Proyecto-Recorrido-del-Caballo-en-Ajedrez

Pagina de Render (La pagina tarda en cargar la primera ves): https://recorrido-del-caballo.onrender.com